

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Diseño de interfaces				CURSO: 2º
	PROTOCOLO:	Examen	AVAL:	2ª	DATA:	13/2/2026
	Nombre y Apellidos:					

NOTAS:

- Crea una solución tipo Biblioteca de Controles de Windows Forms (**.Net Framework**) con el nombre **ApellidosNombreDISF**.
- Añade tu nombre y apellidos al principio de cada archivo.
- Es requisito imprescindible obtener una puntuación mínima de 1.5 en el componente para sumar la nota global del examen.
- Las excepciones tratadas si no se indica lo contrario pueden ser genéricas (clase Exception).
- Pregunta al profesor antes de realizar más métodos de los aquí expuestos o cambiar la estructura indicada en la especificación (por defecto no se puede y podría afectar a la nota).
- Además no se permite el uso de funciones no estudiadas ni tampoco el uso de tipos indefinidos como dynamic. Si tienes dudas pregunta al profesor.

Los juegos de cartas están sobrevalorados, son un hype actual y estamos aburridos y hasta el gorro de tanta historia con las cartitas. Que si Clash Royale, que si Gwent, que si Balatro... ipues no! No puede ser, vamos a meternos a un juego más original, y aunque no sea especialmente novedoso si va a salir del sistema habitual de juegos de cartas. ¡Vamos a programa el piedra, papel, tijera! ¿El de siempre? ¡No! Uno especial que gestionaremosicon cartas!... original ¿eeeeehhhh? (codo, codo, guiño, guiño). Y así le damos experiencia a cada una de las cartas que tengamos ¡Vivan las cartas! ¡JA! ¡¡Cómo molan las cartas!! No se quién puede opinar de otra forma... parfavaaaaaar.

En fin agárrate a las teclas que nos vamos de viaje.

Recuerda: Crea una solución tipo **Biblioteca de Controles de Windows Forms (.Net Framework)** con el nombre **ApellidosNombreDISF**.

1ª Parte Componente.

Se desea desarrollar un componente denominado **Carta** (Archivo *Carta.cs*) que simule una carta de personaje de juego con información de nivel y una barra creciente para indicar un valor o una experiencia. **Puedes ver un ejemplo al final del enunciado.**

Tendrá las siguientes características: Hereda de **UserControl** (cambia ya el nombre al `UserControl1` que sale por defecto en el explorador de soluciones) y contendrá un **PictureBox** denominado **pb** y una etiqueta denominada **lbl**. El componente podrá tener cualquier tamaño.

(0.3p) Las distintas propiedades y eventos definidas a lo largo de este componente deben tener atributos de descripción y pertenecer a una categoría que se denominará Examen. Además se establecerá como evento por defecto del componente `AlcanzaMax` (Definido más abajo).

(1p) El *PictureBox* estará situado en la posición 0,0 del componente y ocupa todo el ancho del mismo y todo el alto menos la altura de **lbl**. Tendrá color de fondo blanco. La imagen, si la hay, se ajustará siempre al tamaño del mismo aunque se deforme.

La etiqueta estará pegada al margen izquierdo y justo debajo de **pb**. Pon su fuente en negrita (*Bold*) y el fondo de un color claro pero distinto del blanco y del gris de fondo del componente (Por ejemplo `LightCyan`). Deja el tamaño de fuente por defecto.

(1.5p) El componente *Carta* tendrá las siguientes propiedades públicas:

- **Imagen:** Da acceso a la propiedad `Image` de **pb**.
- **Nivel:** De tipo entero. Cuando se asigne un valor, además de guardarse en la variable privada se mostrará en **lbl**.
- **Valor:** De tipo entero. Será el porcentaje de la barra. Es decir, puede tener un valor entre 0 y 100. Si es mayor que 100 se queda en 100, si es menor que 0 se queda en 0.

(1p) Se definirá el evento **AlcanzaMax** con delegado *EventHandler* y función *OnAlcanzaMax* estándar que será lanzado cuando *Valor* alcance el nivel 100 pero solo si además la propiedad *Enabled* del componente esté a `true`.

(0.5p) Se desea también que cuando se produzca el evento *DoubleClick* sobre **lbl**, se lance el *DoubleClick* de *Carta*.

(1p) En la parte inferior, a la derecha de la etiqueta, habrá una barra por defecto de color azul. Dibújala con *FillRectangle*. Su altura será la misma que la altura de **lbl**. Será creciente hacia la derecha según el dato *Valor* (si cambia *Valor* cambia la barra).

Si *Valor* es 0, no se ve nada. Si es 100 la barra ocupará todo el ancho desde la etiqueta hasta el final del componente. Cualquier valor intermedio ocupará la parte correspondiente de esa zona del componente (Por ejemplo si *valor* es 50, ocupa la mitad).

	RAMA:	Informática	CICLO:	DAM			
	MÓDULO	Diseño de interfaces				CURSO:	2º
	PROTOCOLO:	Examen	AVAL:	2ª	DATA:	6/3/2020	
	Nombre y Apellidos:						

2ª Parte: Aplicación.

Crea **en la misma solución** un proyecto denominado ***PiedraPapelTijeraCards*** del tipo Windows Forms (.Net Framework).

El formulario principal (Form1) dispondrá de tres componentes Carta a la izquierda, cada uno con un RadioButton debajo (sin texto).

Inicializa las cartas con cada una de las imágenes que se entregan, con Nivel a 1 y Valor a 0.

Añádelas a un vector de forma que en la posición 0 está la carta de piedra, en la 1 la de papel y en la 2 la de tijera.

Dispón de un segundo vector con las imágenes o con los nombres de los archivos de imagen. Si las has cargado previamente en el diseñador, las tienes como recursos (Properties.Resources.NombreImagen). Puedes ver un ejemplo de este código al final del enunciado.

Ten una carta con el radiobutton marcado por defecto. En la medida de lo posible usa el vector junto con los tags de los radiobutton (guarda el índice del vector) si te hicieran falta para gestionar de forma más cómoda estos componentes evitando repetir código y usando los eventos adecuados.

A la derecha del formulario habrá un componente Carta más, que será la carta que vaya sacando la CPU. En principio déjalo vacío, se rellenará cuando se juegue.

A la derecha de dicha carta habrá un ListBox donde se verá de forma textual resultados de jugadas.

Añade también un botón Jugar y una etiqueta donde se indicará el resultado de cada partida.

En la parte inferior habrá un panel de altura 200, que ocupe aproximadamente el ancho del formulario. El fondo será de un color distinto al formulario para que se vea aunque esté vacío.

Las imágenes a usar o las pones como recursos del proyecto o estarán situadas en el subdirectorio **images** dentro de la variable de entorno **userprofile**.

(0.5p) El usuario podrá elegir mediante los radiobuttons cual es la carta deseada para jugar. Luego pulsa jugar y sucede lo siguiente:

(0.7p) La CPU saca un número aleatorio entre 0 y 2 que corresponderá a una carta como se indicó arriba. Pon en la carta de CPU la imagen que corresponda.

El nivel de la carta de la CPU será aleatorio pero el mismo que tenga el jugador en la suya seleccionada ± 1 . Es decir, si por ejemplo el nivel de carta del jugador es 5, la CPU sacará una carta de nivel 4, 5 o 6 de forma aleatoria. El Valor siempre sera 0.

(0.5p) La condición para ganar el jugador será: si el jugador tiene piedra y la cpu tijera, o que el jugador tenga papel y la cpu piedra o que el jugador tenga tijera y la cpu papel. Además el nivel de la carta del jugador debe ser el mismo o mayor que el de la CPU. Se recomienda hacer la comparación con números.

La condición para ganar la CPU es la misma pero a la inversa. Una vez hechas estas comprobaciones en la etiqueta se indica si gana el Jugador o gana la CPU.

Cualquier otro caso se considera empate. Pon el resultado en la etiqueta de resultado.

(0.8p) En un listbox en la que no se permite seleccionar nada irán añadiéndose los resultados de las jugadas. Simplemente el texto que aparece en la etiqueta.

En su tooltip se verá el n.º de veces que ha ganado la CPU.

(0.7p) Si se pulsa F9 se borran de dicha lista todos los empates.

(0.5p) Tras el juego se genera un número aleatorio entre 10 y 20 con el que se incrementa el Valor de la carta seleccionada.

Cuando sea lanzado el evento AlcanzaMax de la carta, se incrementa el Nivel de la misma en 1 y se pone el Valor a 0.

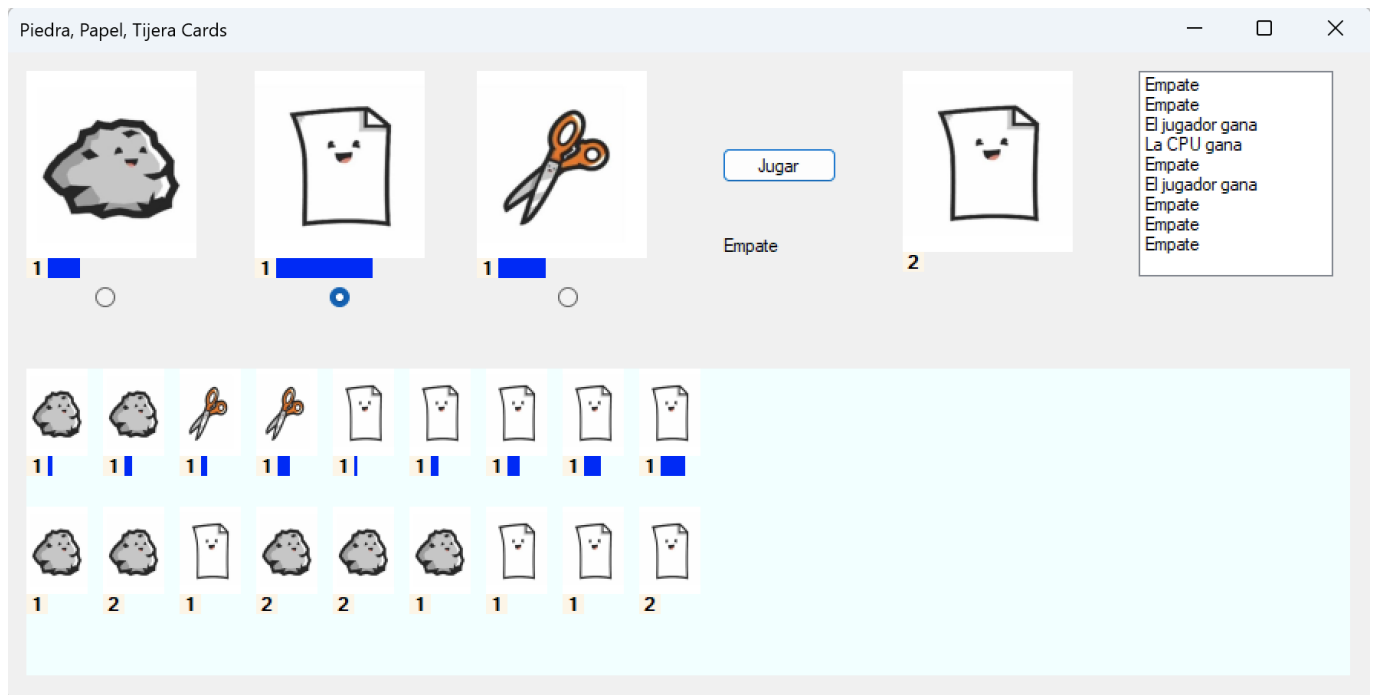
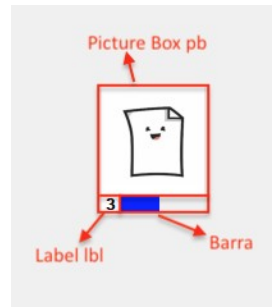
(1p) Finalmente en el panel se guardará un histórico de las jugadas mostrando cartas de ambos jugadores de la siguiente forma:

Partiendo de la posición 0,0 dentro del panel, en dicha posición se coloca una copia de la carta del jugador de esta última jugada y en la posición 0,90 una copia de la del ordenador. El tamaño de cada carta será 40 de ancho por 70 de alto. Luego simplemente cada nueva jugada se desplaza 50 pixels a la derecha. Si quieres haz un constructor a medida en Carta o una función que devuelva una carta con estas características, como lo veas más cómodo. Activa el AutoScroll del panel.

Nota: Si no has hecho el componente completo o no te funciona, crea otro que denominas CartaSimple que contenga el PictureBox y la Label, hazlos públicos, y ponlos en posiciones fijas y sin preocuparte de tamaños. En el PictureBox pondrás las imágenes y en la label informas del Nivel y del Valor.

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Diseño de interfaces				CURSO: 2º
	PROTOCOLO:	Examen	AVAL:	2ª	DATA:	6/3/2020
	Nombre y Apellidos:					

Diseño de componente y Aplicación



Ejemplo de vectores iniciales

```
public partial class Form1 : Form
{
    Carta[] cartas;
    Image[] imagenes;
    public Form1()
    {
        InitializeComponent();
        cartas = new Carta[] { cartaPiedra, cartaPapel, cartaTijera };
        imagenes = new Image[] { Properties.Resources.piedra,
                                Properties.Resources.papel,
                                Properties.Resources.tijera };
    }
}
```